

NANYANG TECHNOLOGICAL UNIVERSITY
SINGAPORE

SC2006 – Software Engineering

Lab 4 Deliverables - Test Cases and Testing Results

Lab Group SCEF

Team 4

Team: Ian Tan, Robin Teoh, Nicholas Ng, Edwin Tan Qi Lun, Benjamin Ng

3.2 Testing / design test cases

3.2.1 Design test cases using equivalence class and boundary value testing techniques to test one important control class

ApplicationController

ApplicationController manages student applications, including submission, withdrawal and preference ranking. It checks the application period, student eligibility, project capacity, attached documents and duplicate applications before saving.

In the current implementation, this logic is in api/app/routers/applications.py. We selected the existing apply() function and tested it through POST /applications.

Black box testing

1) Equivalence class testing

We grouped inputs that should produce the same type of result. For each invalid test, only one input is invalid while all other inputs were kept valid.

Valid inputs:

- an active student with a complete profile
- still eligible for matching
- without an accepted project or unanswered offer
- The project exists, is published and has a free slot
- The submission is within the application period
- the ‘Statement of interest’ has at least 50 characters
- no duplicate application exists
- Documents may be omitted or selected from the student's own profile
- Zero or several existing applications

Invalid inputs:

- Submission before or after the application period
- a Statement of Interest shorter than 50 characters, including an empty string
- an unknown, closed, draft or fully allocated project
- an incomplete student profile
- a student who is ineligible, already has a project placement or has an unanswered offer
- a duplicate application
- a document belonging to another student or an unknown document ID
- submission by an inactive student, a supervisor or an unsigned-in user

2) Boundary value testing

For numeric boundaries, we tested values just below, on and just above the boundary. The application window includes its start and end. All times below are in Singapore time.

Input	Boundary values	Test numbers
Application start	5 Sep 2026: 16:59:59, 17:00:00, 17:00:01	1, 2, 3
Application end	10 Sep 2026: 16:59:58, 16:59:59, 17:00:00	4, 5, 6
Statement Of Interest length	49, 50 and 51 characters	7, 2, 3
Free slots	0, 1 and 2 slots	9, 5, 2

Application test cases and results

Common valid inputs:

- An active, signed-in student (stu-1) with 2 other submitted applications in Round 2
- A student with a complete profile
- Published project listing (lst-8), with 2 total slots and 0 filled slots
- A student eligible for matching
- No accepted project placement or unanswered offer
- No optional supporting documents attached
- Each test starts with fresh test data
- A 100-character Statement of Interest
- Submission of application time = 7 Sep 2026, 18:00 SGT
- No existing application for the same student, listing and round

Unless otherwise stated, each test uses these common valid inputs. Only the changes listed below apply. Test data is prepared independently to check each validation rule, including states that may not normally occur together. Output descriptions summarise the API responses

Numbers in brackets are **HTTP status codes** returned by the API:

- 401 means authentication is required
- 403 means access is denied
- 404 means the requested resource is not found or available
- 409 means the request conflicts with the current system state
- 422 means the request fails validation

Application window: 5 Sep 2026, 17:00:00 SGT to 10 Sep 2026, 16:59:59 SGT, inclusive.

No.	Test input	Expected output	Actual output	Pass ?
1	Submission time = start - 1 second (5 Sep, 16:59:59 SGT).	Application rejected. The application window is closed. (422)	Application rejected. The application window is closed. (422)	Yes
2	Submission time = start; Statement of Interest = 50 characters; Attached documents = [doc-stu-1-cv] (own).	Application saved with preference rank 3. Own document attached.	Application saved with preference rank 3. Own document attached.	Yes
3	Submission time = start + 1 second; Statement of Interest = 51 characters.	Application saved with preference rank 3.	Application saved with preference rank 3.	Yes
4	Submission time = end - 1 second (10 Sep, 16:59:58 SGT).	Application saved with preference rank 3.	Application saved with preference rank 3.	Yes
5	Submission time = end; 2 total slots, 1 filled slot	Application saved with preference rank 3.	Application saved with preference rank 3.	Yes
6	Submission time = end + 1 second (10 Sep, 17:00:00 SGT).	Application rejected. The application window is closed. (422)	Application rejected. The application window is closed. (422)	Yes
7	Statement of Interest = 49 characters	Application rejected. A statement of at least 50 characters is required. (422)	Application rejected. A statement of at least 50 characters is required. (422)	Yes
8	Statement of Interest = empty string; all other inputs valid.	Application rejected. A statement of at least 50 characters is required. (422)	Application rejected. A statement of at least 50 characters is required. (422)	Yes
9	2 total slots, 2 filled slots	Application rejected. The project is full. (409)	Application rejected. The project is full. (409)	Yes
10	listing_id = does-not-exist.	Project not found. (404)	Project not found. (404)	Yes
11	lst-8 status = closed	Project unavailable for applications. (404)	Project unavailable for applications. (404)	Yes
12	lst-8 status = draft	Project unavailable for applications. (404)	Project unavailable for applications. (404)	Yes
13	stu-1 profile is incomplete.	Application rejected. The profile is incomplete. (409)	Application rejected. The profile is incomplete. (409)	Yes
14	stu-1 has an accepted Round 1 match	Application rejected. The student already has a project. (409)	Application rejected. The student already has a project. (409)	Yes
15	stu-1 is not eligible for matching	Application rejected. The student is not eligible. (409)	Application rejected. The student is not eligible. (409)	Yes
16	stu-1 has a published Round 1 offer with a response deadline that has not passed	Application rejected. The student has an unanswered offer. (409)	Application rejected. The student has an unanswered offer. (409)	Yes

No.	Test input	Expected output	Actual output	Pass ?
17	An existing submitted application is for lst-8 in this round.	Application rejected. A duplicate application exists. (409)	Application rejected. A duplicate application exists. (409)	Yes
18	Attached documents = [doc-stu-7-t], belonging to another student.	Application rejected. The document is not in the student's profile. (422)	Application rejected. The document is not in the student's profile. (422)	Yes
19	Attached documents = [does-not-exist].	Application rejected. The document is not in the student's profile. (422)	Application rejected. The document is not in the student's profile. (422)	Yes
20	Existing applications withdrawn in setup; 0 active applications.	Application saved with preference rank 1.	Application saved with preference rank 1.	Yes
21	student (stu-1) has inactive account status.	Access denied. The account is inactive. (403)	Access denied. The account is inactive. (403)	Yes
22	A signed-in supervisor (sup-1) attempts to submit an application	Access denied. Only students can apply. (403)	Access denied. Only students can apply. (403)	Yes
23	An unsigned-in user submits a request without authentication	Access denied. Sign-in is required. (401)	Access denied. Sign-in is required. (401)	Yes

Result: 23 passed, 0 failed. Successful tests also checked the saved application and preference rank. Rejected requests left the application records unchanged.

3.2.2 Design test cases using basis path testing techniques to test two methods that implement complex application logic.

White box testing

Method 1: match(students, listings) *api/app/services/matching/algorithm.py.*

1) **Matching**

It allocates students using their project preferences, supervisor rankings and available capacity.

The control-flow graph (CFG) shows loops, conditional branches and short-circuit checks separately. Data preparation and calls to helper functions are shown as single processing steps.

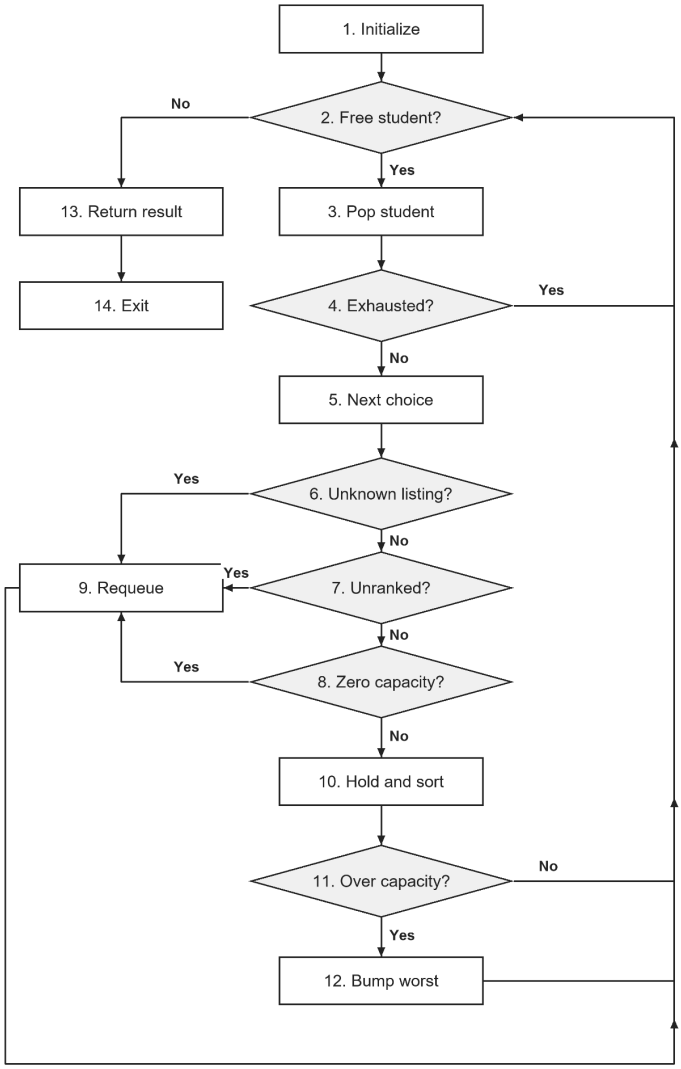


Figure 1 Matching control flow graph

CFG node descriptions

Node	Description	Node	Description
1	Prepare the student preferences, supervisor rankings, project capacities and unmatched-student queue.	8	Check whether the project has zero capacity. **Yes**: Go to node 9 to requeue the student.

			No: Go to node 10 to temporarily hold the student for the project.
2	Check whether any students remain in the queue. **Yes**: Go to node 3 to select a student. **No**: Go to node 13 to return the matching results	9	Return the student to the queue so their next preferred project can be considered. Go back to node 2 to check for students awaiting processing.
3	Remove one student from the queue for processing.	10	Temporarily hold the student for the project and sort its held students by the supervisor's ranking.
4	Check whether all projects on the student's preference list have been considered for matching **Yes**: The student remains unmatched. Return to node 2. **No**: Go to node 5.	11	Check whether the number of held students exceeds the project's capacity. **Yes**: Go to node 12 to remove and requeue the lowest-ranked held student. **No**: Keep the held students and return to node 2.
5	Select the student's next preferred project.	12	Remove the lowest-ranked held student and return them to the queue.
6	Check whether the selected project is missing from the supplied project listings. **Yes**: Go to node 9 to requeue the student. **No**: Go to node 7 to check whether the supervisor ranked the student.	13	Build and return the final project assignments and list of unmatched students.
7	Check whether the supervisor has omitted the student from the project's applicant ranking. **Yes**: Go to node 9 to requeue the student. **No**: Go to node 8 to check project capacity.	14	End the method.

Matching basis path testing

Cyclomatic complexity :

decision points + 1 = 6 + 1 = **7**

edges − nodes + 2 = 19 − 14 + 2 = **7**

We therefore selected the following **7 independent basis paths**:

1. **Baseline path:** 1, 2, 13, 14
2. **Basis path 2:** 1, 2, 3, 4, 2, 13, 14
3. **Basis path 3:** 1, 2, 3, 4, 5, 6, 9, 2, 3, 4, 2, 13, 14
4. **Basis path 4:** 1, 2, 3, 4, 5, 6, 7, 9, 2, 3, 4, 2, 13, 14
5. **Basis path 5:** 1, 2, 3, 4, 5, 6, 7, 8, 9, 2, 3, 4, 2, 13, 14
6. **Basis path 6:** 1, 2, 3, 4, 5, 6, 7, 8, 10, 11, 2, 13, 14
7. **Basis path 7:** 1, 2, 3, 4, 5, 6, 7, 8, 10, 11, 2, 3, 4, 5, 6, 7, 8, 10, 11, 12, 2, 3, 4, 2, 13, 14

Each additional basis path introduces at least one edge not covered by the preceding paths. Test 7 checks that, when capacity is exceeded, the project retains the student ranked higher by the supervisor and requeues the other student.

Test cases and results

No.	Test input	Expected output	Actual output	Pass?
1	No students and no projects.	No matches.	No matches.	Yes
2	Student 1 has no project preferences.	No matches. Student 1 unmatched.	No matches. Student 1 unmatched.	Yes
3	Student 1 chooses Project 1, but the project does not exist.	No matches. Student 1 unmatched.	No matches. Student 1 unmatched.	Yes
4	Student 1 chooses Project 1, but is not ranked by its supervisor.	No matches. Student 1 unmatched.	No matches. Student 1 unmatched.	Yes
5	Student 1 chooses Project 1 and is ranked. Capacity = 0.	No matches. Student 1 unmatched.	No matches. Student 1 unmatched.	Yes
6	Student 1 chooses Project 1 and is ranked. Capacity = 1.	Student 1 matched to Project 1.	Student 1 matched to Project 1.	Yes
7	Students 1 and 2 choose Project 1. Capacity = 1. The supervisor prefers Student 1.	Student 1 matched to Project 1. Student 2 unmatched.	Student 1 matched to Project 1. Student 2 unmatched.	Yes

Method 2: withdraw(application_id, user, store, settings)

api/app/routers/applications.py.

2) <u>Application withdrawal</u>

It checks the application owner and status, withdraws the application and updates the remaining preference ranks.

The paths assume an active student and an open application period. The application-period check and the collection of remaining applications are each shown as one step, without their internal details.

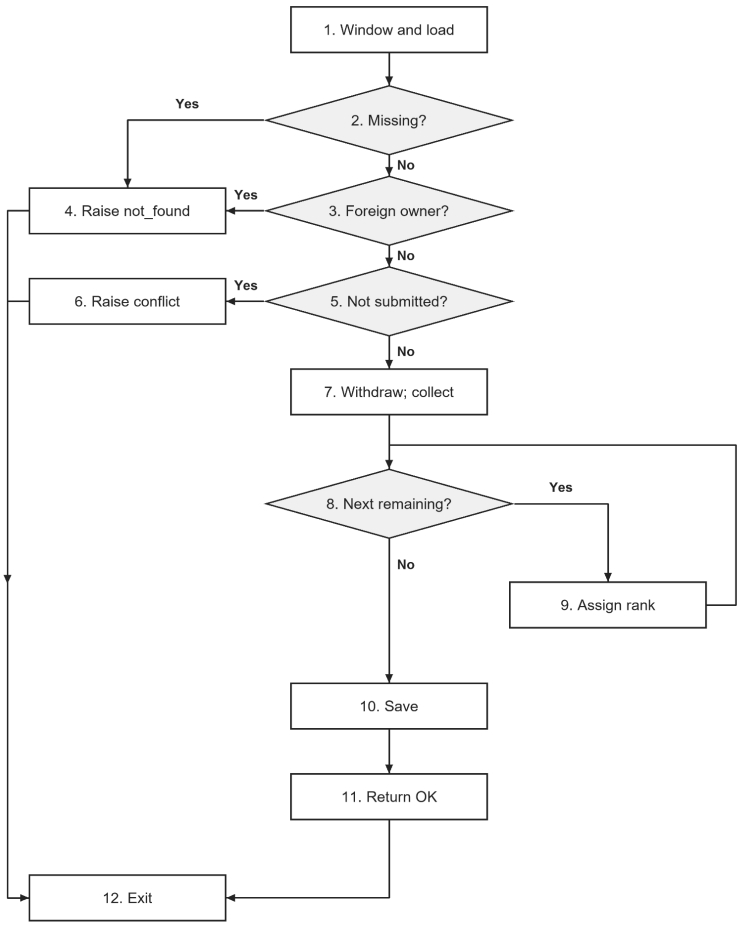


Figure 2 Withdrawal control flow graph

CFG node descriptions

Node	Description	Node	Description
1	Check the application period and retrieve the application using its ID.	**7**	Mark the application as withdrawn and clear its preference rank. Collect the student's remaining submitted applications in the same round, sorted by their current ranks.
2	Check whether the application is missing. **Yes:** Go to node 4 to raise a not-found error. **No:** Go to node 3 to check ownership.	**8**	Check whether another remaining application needs its rank updated. **Yes:** Go to node 9 to assign its new rank. **No:** Go to node 10 to save the changes.

3	Check whether the application belongs to another student. **Yes:** Go to node 4 to raise a not-found error. **No:** Go to node 5 to check the application's status.	**9**	Assign the next sequential preference rank, starting from 1. Return to node 8 to check for another remaining application.
4	Raise an application-not-found error (404). Go to node 12 to exit without changing application records.	**10**	Save the withdrawn application and the updated preference ranks of the remaining applications.
5	Check whether the application's status is anything other than submitted. Yes: Go to node 6 to reject the withdrawal. No: Go to node 7 to withdraw the application.	**11**	Return a successful response (200) containing {"ok": true}.
6	Raise a conflict error (409), because only submitted applications can be withdrawn. Go to node 12 to exit without changing application records.	**12**	End the method.

Withdrawal basis path testing

Cyclomatic complexity:

decision points + 1 = 4 + 1 = **5**

edges − nodes + 2 = 15 − 12 + 2 = **5**

We therefore selected the following **5 independent basis paths**:

1. **Baseline path:** 1, 2, 4, 12
2. **Basis path 2:** 1, 2, 3, 4, 12
3. **Basis path 3:** 1, 2, 3, 5, 6, 12
4. **Basis path 4:** 1, 2, 3, 5, 7, 8, 10, 11, 12
5. **Basis path 5:** 1, 2, 3, 5, 7, 8, 9, 8, 9, 8, 10, 11, 12

Test cases and results

Unless otherwise stated, Student 1 has three submitted applications in the same round, ranked 1, 2 and 3. In Test 4, only the target application remains submitted.

No.	Test input	Expected output	Actual output	Pass?
1	Application ID does not exist.	Application not found. (404)	Application not found. (404)	Yes

No.	Test input	Expected output	Actual output	Pass?
2	Student 1 is trying to withdraw Student 7's application	Application not found. (404)	Application not found. (404)	Yes
3	The student's application has already been matched.	Withdrawal rejected. The application is no longer under review. (409)	Withdrawal rejected. The application is no longer under review. (409)	Yes
4	Withdraw the student's only active application.	Application withdrawn. No active applications remain.	Application withdrawn. No active applications remain.	Yes
5	Withdraw rank 1 from three active applications ranked 1, 2 and 3.	Application withdrawn. Other ranks become 1 and 2.	Application withdrawn. Other ranks become 1 and 2.	Yes

Tests 4 and 5: The withdrawn application's rank is cleared because it is no longer an active choice.

Test 5: After withdrawing rank 1, the application previously ranked **2 becomes 1**, and the one ranked **3 becomes 2**.

3.2.3 Minimize the number of test cases

We reduced the number of tests by combining valid conditions. For example, application Test 2 checks the opening time, a 50-character statement and the student's own document together. Invalid conditions were tested one at a time so we could identify the reason for rejection.

For white-box testing, we selected 7 matching basis paths and 5 withdrawal basis paths according to their cyclomatic complexity, avoiding the need to test every possible loop path.

Tests used fresh in-memory test data. Submission and withdrawal tests used FastAPI TestClient, while matching tests called the algorithm directly. Expected and actual results are recorded in the test tables.